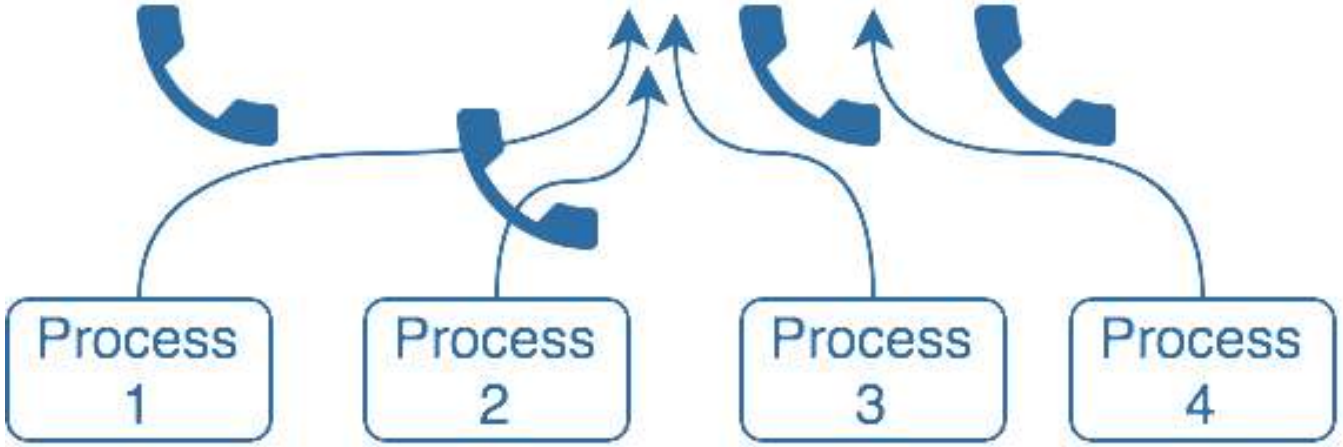
# System Calls

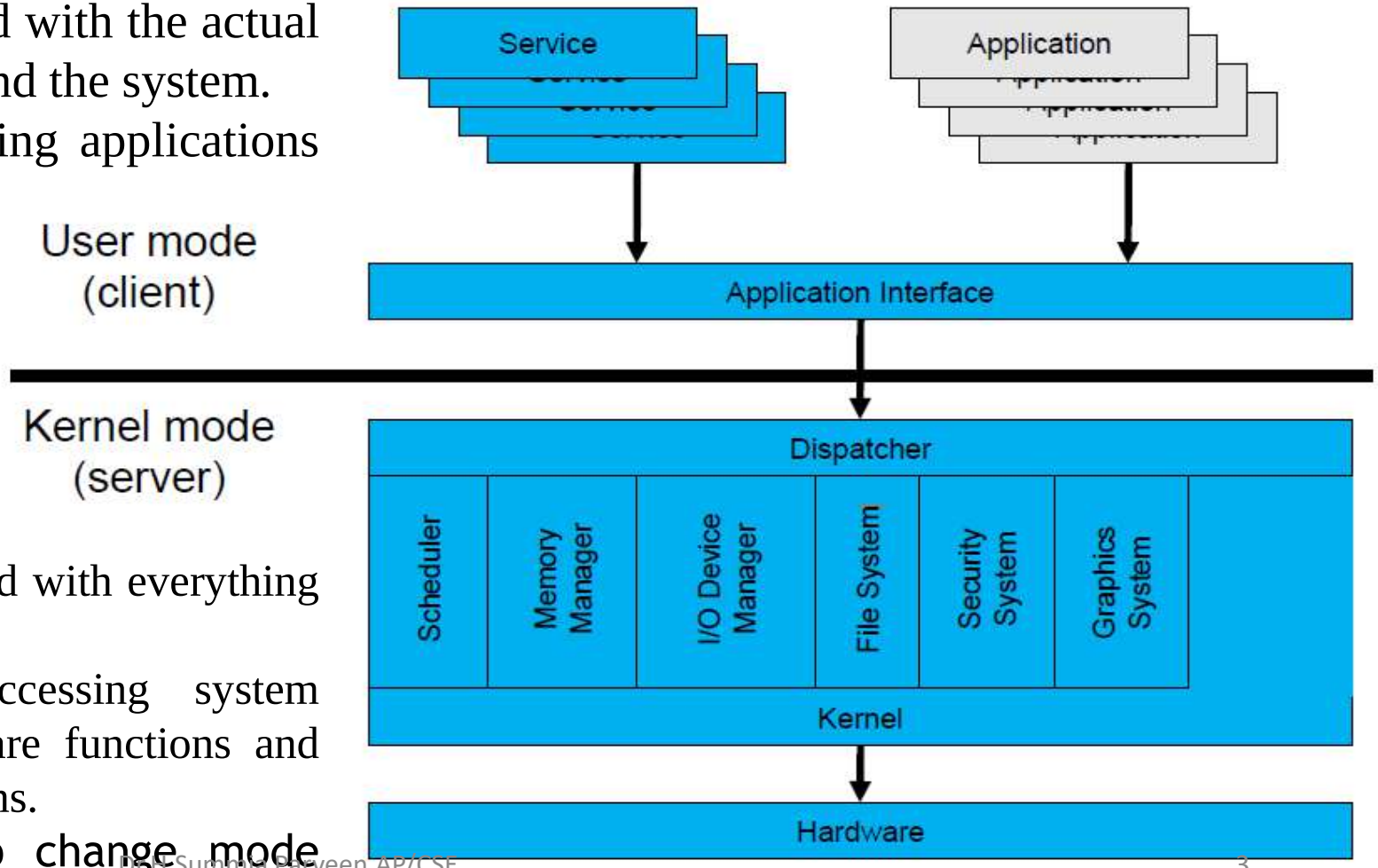


Maid
(Kernel)

Homemaker
(Operating System)

# Operating System Mode

- The *User Mode* is concerned with the actual interface between the user and the system.
- It controls things like running applications and accessing files.




- The *Kernel Mode* is concerned with everything running in the background.
- It controls things like accessing system resources, controlling hardware functions and processing program instructions.
- System calls are used to change mode from User to Kernel.

# Kernel

- Kernel is a software code that reside in central core of OS. It has complete control over system.
- When operation system boots, kernel is first part of OS to load in main memory.
- Kernel remains in main memory for entire duration of computer session. The kernel code is usually loaded in to protected area of memory.
- Kernel performs it’s task like executing processes and handling interrupts in kernel space.
- User performs it’s task in user area of memory.
- This memory separation is made in order to prevent user data and kernel data from interfering with each other.
- Kernel does not interact directly with user, but it interacts using SHELL and other programs and hardware.

# Kernel cont…

- Kernel includes:-

  1. Scheduler: It allocates the Kernel's processing time to various processes.

  2. Supervisor: It grants permission to use computer system resources to each process.

  3. Interrupt handler : It handles all requests from the various hardware devices which compete for kernel services.

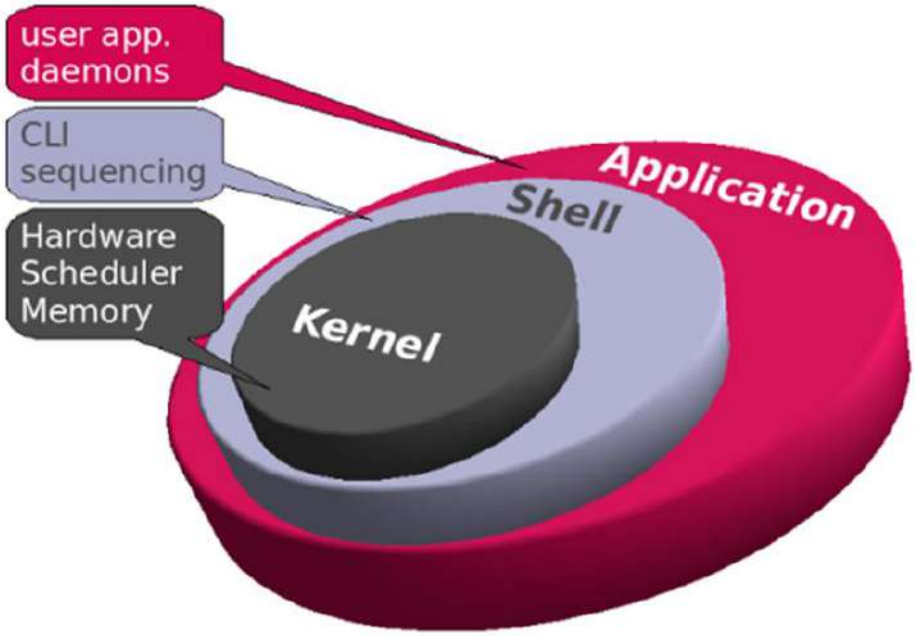
  4. Memory manager : allocates space in memory for all users of kernel service.

- kernel provides services for process management, file management, I/O management, memory management.
- System calls are used to provide these type of services.

# KERNEL VERSUS SHELL

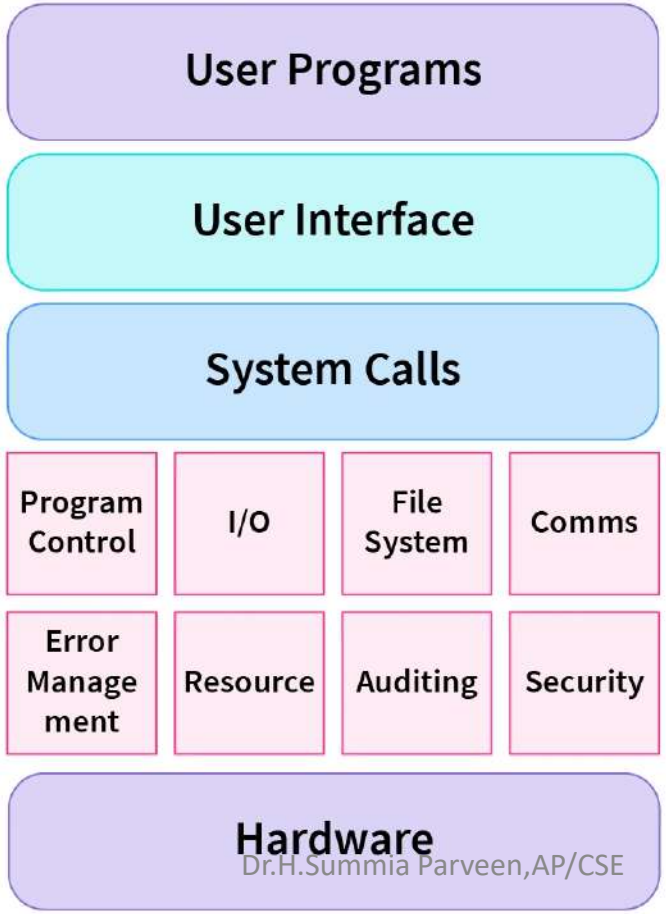
| KERNEL | SHELL |
| --- | --- |
| A computer program which acts as the core of the computer's operating system and has the control over everything in the system | A computer program which works as the interface to access the services provided by the operating system |
| Core of the system that controls all the tasks of the system | Interface between the kernel and user |
| Does not have types | Has types such as Bourne shell, C shell, Korn Shell, Bourne Again Shell, etc. |

Visit www.PEDIAA.com



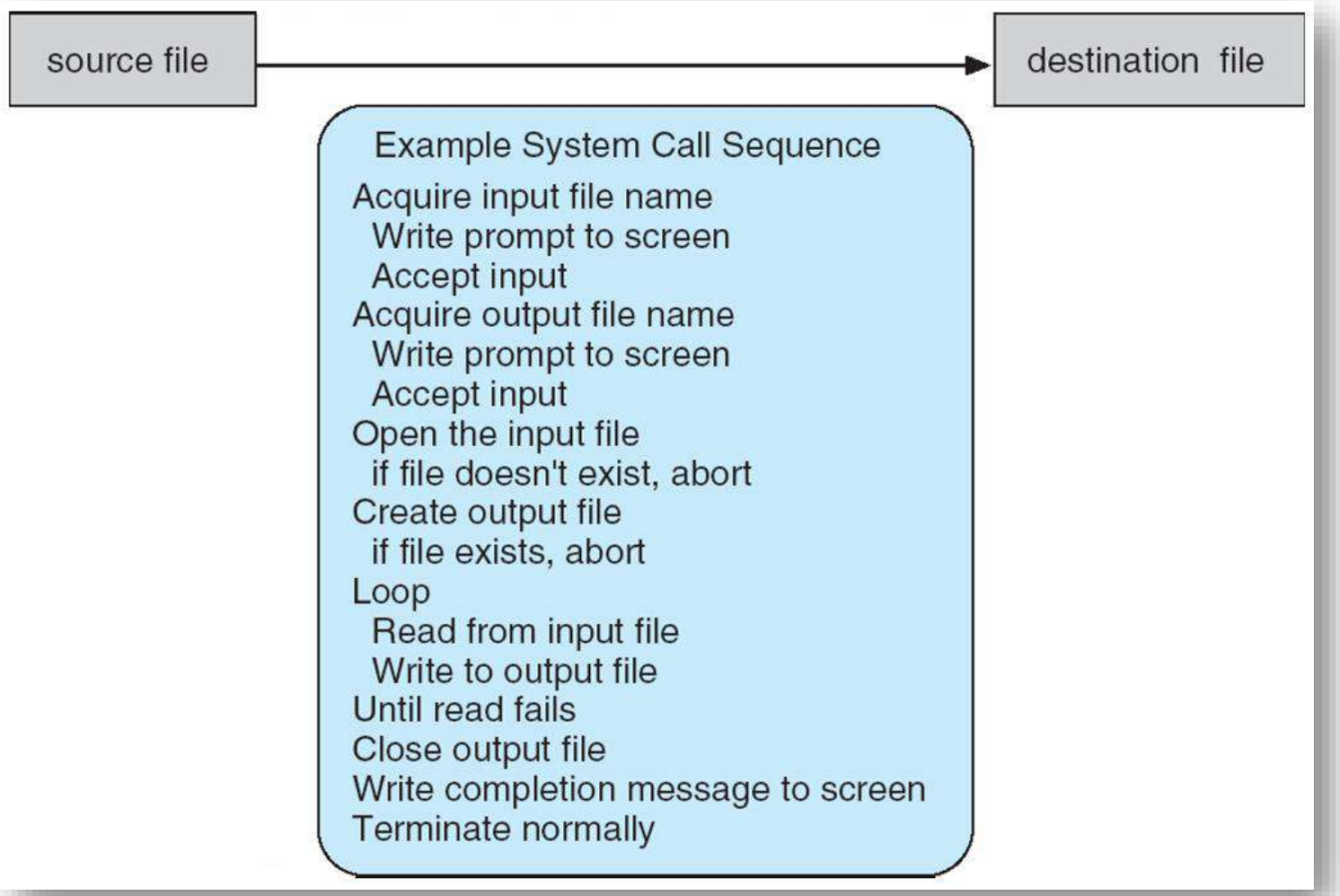
# System Calls

- System calls provide the interface between a process (User program or application software) and the operating system.
- System call is how a program requests a service from an operating system's kernel.
- These calls are generally available as assembly-language instructions.



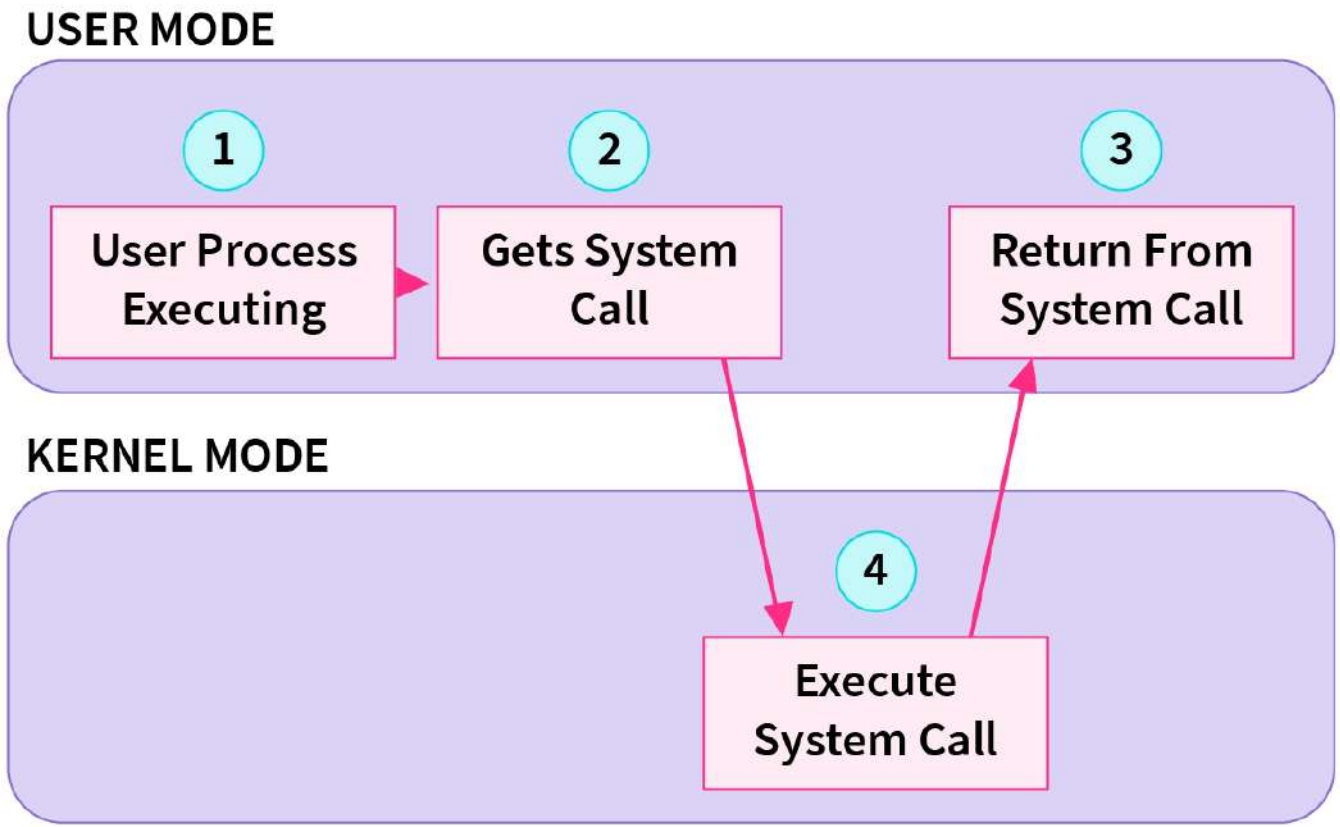
# Example of System Calls

- **System call sequence to copy the contents of one file to another file**



# How the System Call works?

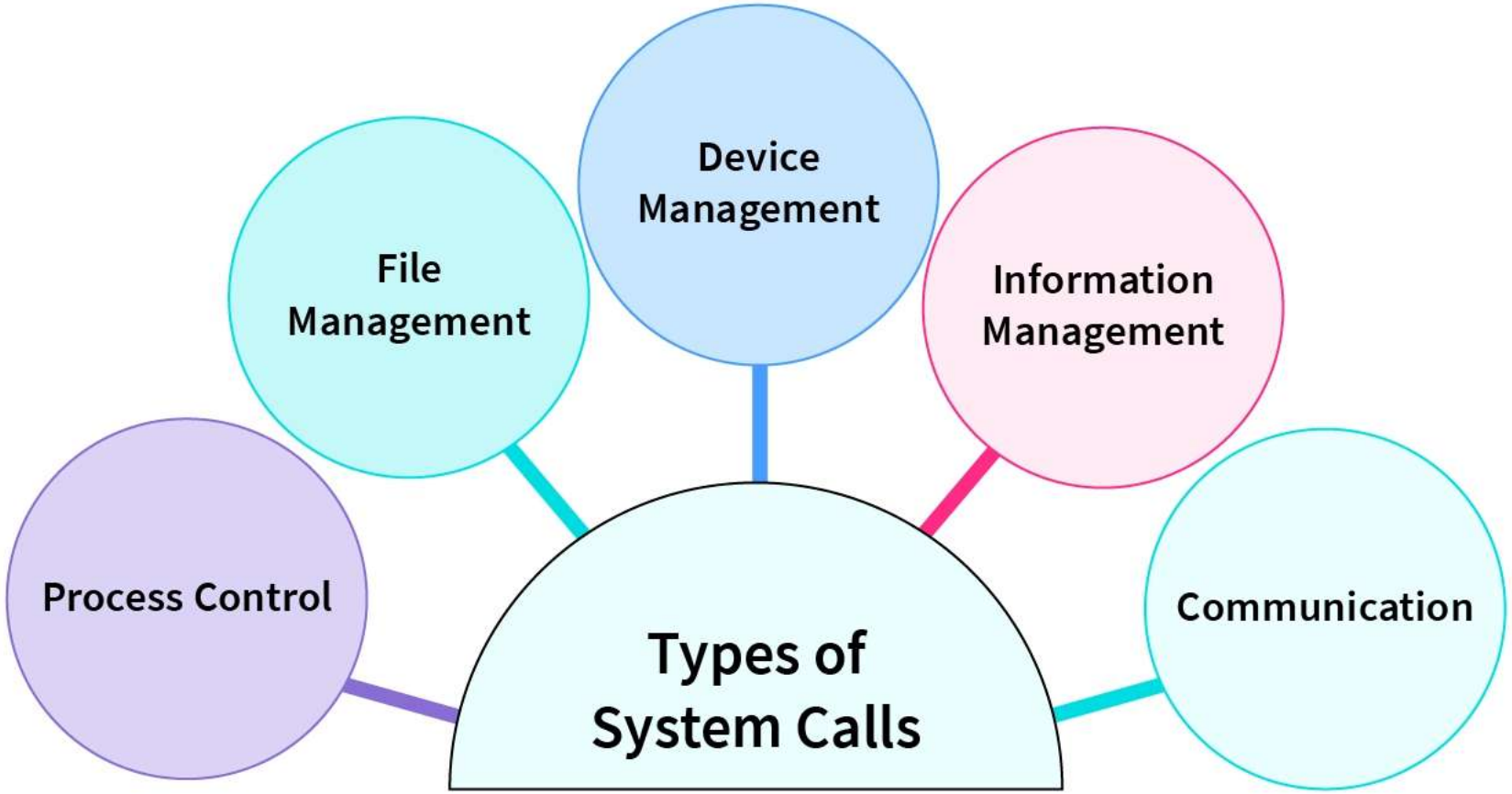
1. **User Mode:** In this mode, execution is done on behalf of the user.
2. **Monitor/Kernel-Mode:** In this mode, execution is done on behalf of OS



# How the System Call works?

- When the process is under execution process executes under user mode, but when the process requires some os resources to complete the task, the process makes a system call.
- **System call is executed in kernel mode on a priority basis**.
- On completion of the system call, the control is passed to process in user mode.
- The process continues the execution in user mode.
- System call is made by sending a trap signal to the kernel, which reads the system call code from the register and executes the system call.

# Types of System Calls



# Types of System Calls

- **File management**
  - **create file, delete file**
  - **open, close file**
  - **read, write, reposition**
  - **get and set file attributes**
- **Device management**
  - **request device, release device**
  - **read, write, reposition**
  - **get device attributes, set device attributes**
  - **logically attach or detach devices**

# Types of System Calls

- **Process control**
  - **create process, terminate process**
  - **end, abort**
  - **load, execute**
  - **get process attributes, set process attributes**
  - **wait for time**
  - **wait event, signal event**
  - **allocate and free memory**
  - **Dump memory if error**
  - **Debugger for determining bugs, single step execution**
  - **Locks for managing access to shared data between processes**

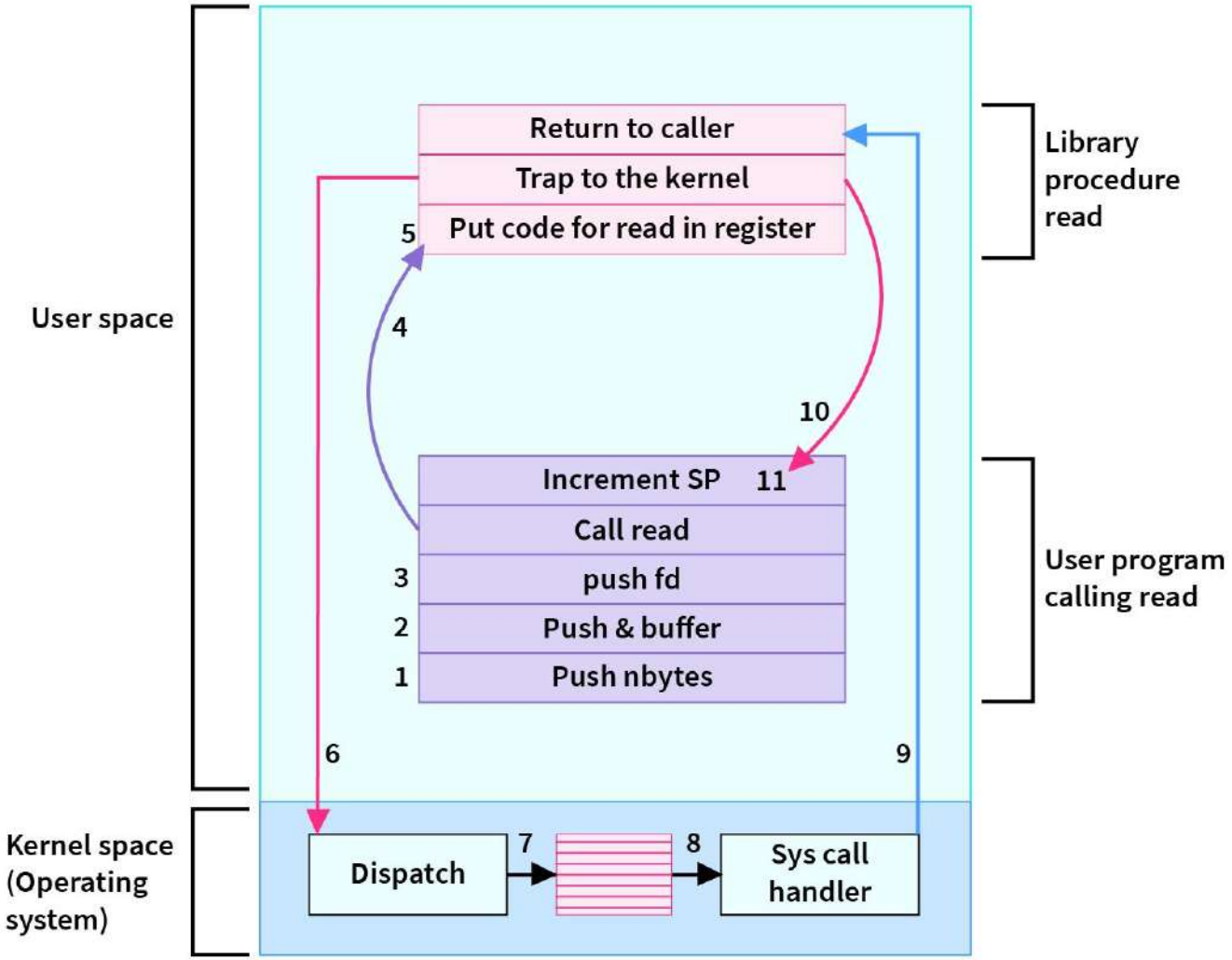
# Types of System Calls

- **Information maintenance**
  - **get time or date, set time or date**
  - **get system data, set system data**
  - **get and set process, file, or device attributes**
- **Communications**
  - **create, delete communication connection**
  - **send, receive messages if message passing model to host name or process name**
    - **From client to server**
  - **Shared-memory model create and gain access to memory regions**
  - **transfer status information**
  - **attach and detach remote devices**

# Types of System Calls

- **Protection**
  - **Control access to resources**
  - **Get and set permissions**
  - **Allow and deny user access**

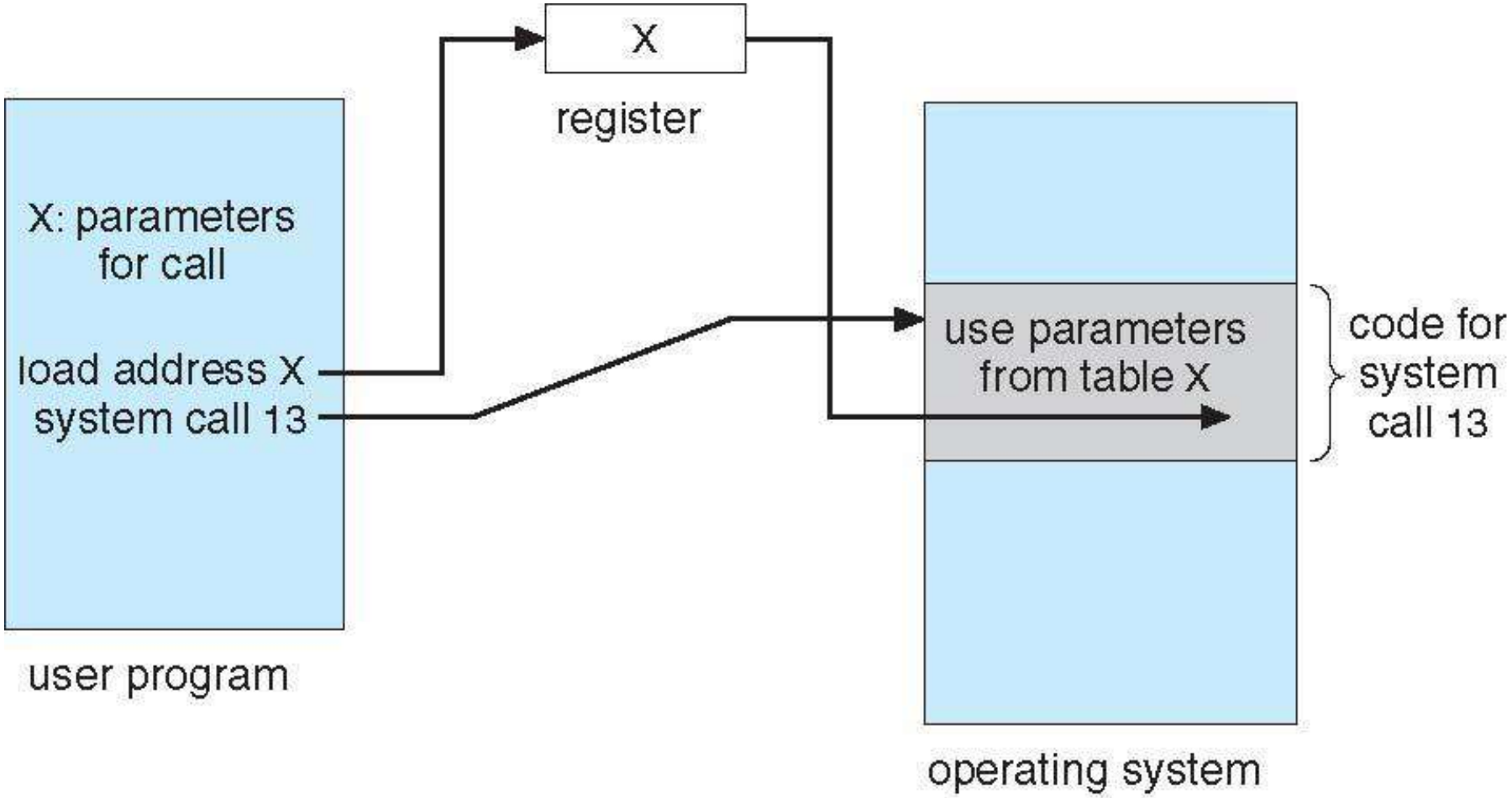
# Example of System Calls : read() system call of file management



# System Call Parameter Passing

- Three general methods used to pass parameters to the OS
  1. Simplest: pass the parameters in registers
     - In some cases, may be more parameters than registers
  2. Parameters stored in a block, or table, in memory, and address of block passed as a parameter in a register
     - This approach taken by Linux and Solaris
  3. Parameters placed, or **pushed**, onto the **stack** by the program and **popped** off the stack by the operating system
  4. Block and stack methods do not limit the number or length of parameters being passed

# Parameter Passing via Table



# Important System Calls

**1.wait():** There are scenarios where the current process needs to wait for another process to complete the task. So to wait in this scenario, wait() system call is used.

**2.fork():** This system call creates the copy of the process which has called it. The one which has called fork is called the parent process and the newly created copy is called the child process.

**3.exec():** This system call is called when the running process wants to execute another executable file. The process id remains the same while the other resources used by the process are replaced by the newly created process.

**4.kill():** Sometimes while working, we require to terminate a certain process. So kill system call is called which sends the termination signal to the process.

**5.exit():** When we are actually required to terminate the program, an exit system call is used. The resources which are occupied by the process are released after invoking the exit system call.

# Examples of Windows and Unix System Calls

| | Windows | Unix |
|---|---|---|
| Process Control | CreateProcess() | fork() |
| | ExitProcess() | exit() |
| | WaitForSingleObject() | wait() |
| File Manipulation | CreateFile() | open() |
| | ReadFile() | read() |
| | WriteFile() | write() |
| | CloseHandle() | close() |
| Device Manipulation | SetConsoleMode() | ioctl() |
| | ReadConsole() | read() |
| | WriteConsole() | write() |
| Information Maintenance | GetCurrentProcessID() | getpid() |
| | SetTimer() | alarm() |
| | Sleep() | sleep() |
| Communication | CreatePipe() | pipe() |
| | CreateFileMapping() | shmget() |
| | MapViewOfFile() | mmap() |
| Protection | SetFileSecurity() | chmod() |
| | InitlializeSecurityDescriptor() | umask() |
| | SetSecurityDescriptorGroup() | chown() |

# System Programs

- System programs provide a convenient environment for program development and execution. They can be divided into:
  - **File manipulation**
  - **Status information sometimes stored in a File modification**
  - **Programming language support**
  - **Program loading and execution**
  - **Communications**
  - **Background services**
  - **Application programs**
- Most users' view of the operation system is defined by system programs, not the actual system calls

# System Programs

- Provide a convenient environment for program development and execution
  - Some of them are simply user interfaces to system calls; others are considerably more complex
- **File management** - Create, delete, copy, rename, print, dump, list, and generally manipulate files and directories
- **Status information**
  - Some ask the system for info - date, time, amount of available memory, disk space, number of users
  - Others provide detailed performance, logging, and debugging information
  - Typically, these programs format and print the output to the terminal or other output devices
  - Some systems implement a **registry** - used to store and retrieve configuration information

# System Programs (Cont.)

- **File modification**
  - Text editors to create and modify files
  - Special commands to search contents of files or perform transformations of the text
- **Programming-language support** - Compilers, assemblers, debuggers and interpreters sometimes provided
- **Program loading and execution**- Absolute loaders, relocatable loaders, linkage editors, and overlay-loaders, debugging systems for higher-level and machine language
- **Communications** - Provide the mechanism for creating virtual connections among processes, users, and computer systems
  - Allow users to send messages to one another's screens, browse web pages, send electronic-mail messages, log in remotely, transfer files from one machine to another

# System Programs (Cont.)

- **Background Services**
  - Launch at boot time
    - Some for system startup, then terminate
    - Some from system boot to shutdown
  - Provide facilities like disk checking, process scheduling, error logging, printing
  - Run in user context not kernel context
  - Known as **services**, **subsystems**, **daemons**
- **Application programs**
  - Don't pertain to system
  - Run by users
  - Not typically considered part of OS
  - Launched by command line, mouse click, finger poke